

UR5e Control Analysis

Part 1A — Kinematic and Dynamic Characterization

Nonlinear Plant Definition, Singularity Analysis, Linearization,
Benchmark Design, and Python-C++ Cross-Validation

Sooyong Kim

Robotics Engineering Portfolio Project

September 2026

Scope. This report begins after the Pre-Part 1 model/simulation setup and analytical-Isaac Sim cross-validation. The objective here is not to rebuild the UR5e model, but to characterize the validated plant sufficiently to support controller design in Part 1B.

Executive Summary

I treated Part 1A as a model-characterization problem rather than a controller-design exercise. Before tuning PID, computed-torque, LQR, or MPC controllers, I wanted to know what the validated UR5e model actually does: where its Jacobian loses numerical robustness, how its generalized inertia and gravity loading change with configuration, how accurate a local linearization remains away from equilibrium, and whether a common benchmark trajectory is dynamically meaningful without being dominated by actuator saturation.

The analysis used the UR5e model previously aligned with the ROS 2/URDF description and cross-validated against Isaac Sim. Kinematics used the standard UR5e DH geometry, while rigid-body dynamics used the physical link-frame transforms, masses, centers of mass, and inertia tensors established during model validation. This distinction was deliberate: a compact DH representation is convenient for forward kinematics and Jacobians, but the dynamic model must preserve the physical link-frame definitions associated with each inertia tensor.

The nominal working configuration $\mathbf{q} = [0, -90, 90, -90, -90, 0]^\circ$ was well conditioned relative to the zero configuration, with a full Jacobian minimum singular value of 0.2224 and condition number 8.31. Along a joint-space interpolation toward $\mathbf{q} = \mathbf{0}$, the minimum singular value approached zero while the condition number rose sharply before rank loss. A Moore–Penrose pseudoinverse consequently amplified the weakest task-space command without bound, whereas damped least squares (DLS) with $\lambda = 0.05$ bounded the amplification near the expected theoretical maximum of 10, at the cost of increasing task-space error.

The dynamic characterization confirmed strong configuration dependence. The mass matrix remained symmetric positive definite at all investigated poses, but its eigenvalue spread and gravity vector changed substantially. For the selected test velocity, gravity dominated the Coriolis/centrifugal term; this is a statement about the specified operating condition, not a general claim that velocity-dependent dynamics are negligible. The Coriolis implementation was additionally checked by explicitly constructing $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$: the maximum discrepancy in $\mathbf{C}\dot{\mathbf{q}} - \mathbf{c}$ was 8.33×10^{-17} , and the maximum residual of the standard rigid-body skew-symmetry identity was 1.44×10^{-9} across the three investigated configurations.

A 12-state nonlinear torque-driven plant was then defined and statically balanced with $\boldsymbol{\tau}_{\text{eq}} = \mathbf{g}(\mathbf{q}_{\text{eq}})$. Numerical linearization around three equilibria produced full controllability rank (12/12). Finally, I defined a 4 s quintic joint-space benchmark sampled at 2 ms. Its maximum reference speed was 7.81% of the 180°s^{-1} joint-speed specification, while inverse-dynamics torque peaked at 40.79 N m on Joint 2, or 27.2% of the adopted simulation torque constraint. A fixed-size Eigen C++ implementation reproduced the Python reference calculations within the specified tolerances.

1. Scope, Model Basis, and Engineering Method

Part 1A starts from a model that has already passed a separate setup and cross-validation stage. In that earlier stage I reconciled the URDF/Isaac frame conventions, fixed-base configuration, gravity, joint drives, physical-link transforms, Jacobian, mass matrix, and gravity vector. The present report therefore asks a different question: once the model is internally and externally consistent, what characteristics matter for controller design?

1.1 Model basis

The kinematic model uses the standard UR5e DH geometry. Universal Robots lists $a = [0, -0.4250, -0.3922, 0, 0, 0]$ m, $d = [0.1625, 0, 0, 0.1333, 0.0997, 0.0996]$ m, and $\alpha = [\pi/2, 0, 0, \pi/2, -\pi/2, 0]$.

Table 1: Standard UR5e DH parameters used for kinematics.

Joint	a [m]	d [m]	α [rad]
J1	0	0.1625	$+\pi/2$
J2	-0.4250	0	0
J3	-0.3922	0	0
J4	0	0.1333	$+\pi/2$
J5	0	0.0997	$-\pi/2$
J6	0	0.0996	0

The standard DH chain is reserved for kinematic calculations such as forward kinematics and the geometric Jacobian. Rigid-body dynamics, however, are evaluated using the validated URDF physical-link frame chain rather than by attaching the identified inertial properties directly to the DH frames. The physical-link transformation chain includes the initial $R_z(\pi)$ alignment associated with `base_link` to `base_link_inertia`, together with the fixed origins and rotations defined by the URDF joint chain. Each link center of mass is expressed in its corresponding physical local frame, and each inertia tensor is rotated into the base frame using the physical-link orientation. The same formulation is used by the Python and C++ implementations. The resulting $\mathbf{M}(\mathbf{q})$ and $\mathbf{g}(\mathbf{q})$ were independently cross-checked against Isaac Sim at the tested configurations during the preceding model-validation stage.

1.2 Dynamic convention

I use the manipulator equation in compensation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau}. \quad (1)$$

With this sign convention, $\mathbf{g}(\mathbf{q})$ is the joint torque required to compensate gravity at zero velocity and acceleration. The state is $\mathbf{x} = [\mathbf{q}^\top \dot{\mathbf{q}}^\top]^\top \in \mathbb{R}^{12}$, and the input is the six-joint torque vector $\boldsymbol{\tau}$.

1.3 Characterization philosophy

- Use representative configurations rather than one convenient pose.
- Separate geometric singularity from numerical conditioning.
- Check physical consistency (e.g., positive-definite $\mathbf{M}$) before interpreting controller implications.
- Quantify the local validity of the linearized model instead of assuming a linear model is globally representative.
- Define one benchmark before controller tuning so every controller is judged on the same reference and constraints.
- Keep Python as the numerical/reference environment and reproduce the core plant in fixed-size C++/Eigen before runtime controller work.

2. Kinematic Characterization

I evaluated three configurations: the validated test pose $\mathbf{q} = [10, -40, 60, -30, 45, 20]^\circ$, the nominal working pose $\mathbf{q} = [0, -90, 90, -90, -90, 0]^\circ$, and the zero configuration.

Table 2: Representative kinematic characterization.

Configuration	TCP position [m]	$\sigma_{\min}(J)$	$\kappa(J)$	rank
Validated	$[-0.73355, -0.33621, 0.21559]$	0.15682	12.95	6
Nominal	$[-0.49190, -0.13330, 0.48790]$	0.22236	8.31	6
Zero	$[-0.81720, -0.23290, 0.06280]$	≈ 0	∞	5

The zero configuration lost one rotational degree of instantaneous motion while retaining rank three in the translational Jacobian. A geometric Jacobian combines translational units (m/s) and rotational units (rad/s), so its full condition number and manipulability are scale-dependent. I use them comparatively under one consistent definition and retain separate J_v/J_ω results where physical interpretation matters.

3. Singularity Approach and Numerical Robustness

To study degradation before exact rank loss, I interpolated in joint space from the nominal working pose ($s = 0$) to the zero configuration ($s = 1$). The minimum singular value decreased continuously from 0.22236 toward zero. The Jacobian remained full rank until the endpoint, but its condition number increased rapidly beforehand.

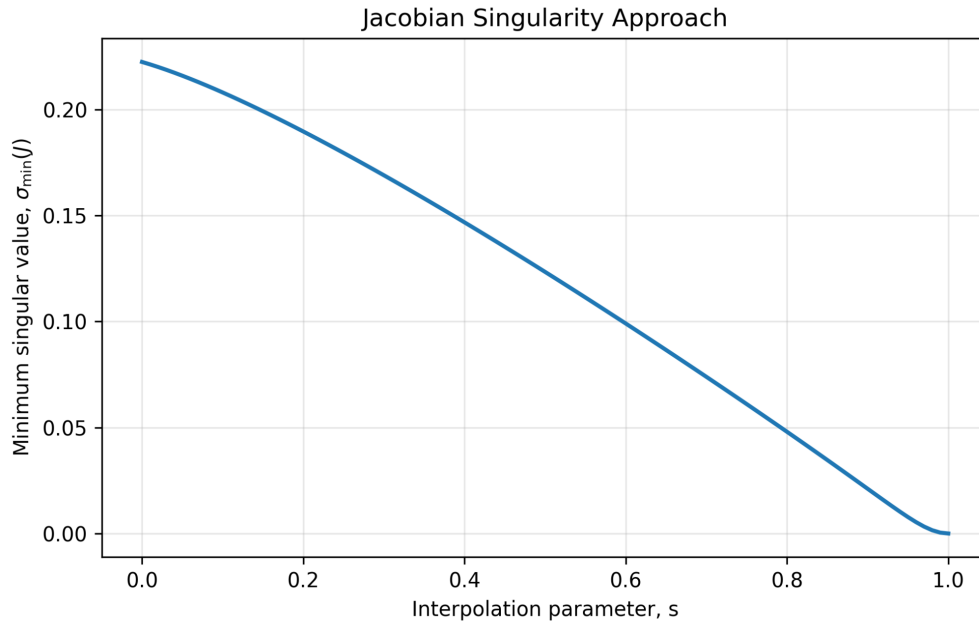


Figure 1: Minimum singular value along the nominal-to-singular joint-space interpolation.

This result changed the way I framed singularity handling for the controller work. A binary rank test is too late to be useful by itself. A controller or inverse-kinematics layer can become numerically aggressive while the matrix is still formally full rank.

4. Pseudoinverse versus Damped Least Squares

I next commanded a unit task-space velocity along the weakest instantaneous left-singular direction of the Jacobian at each interpolated configuration. This deliberately exposes the direction in

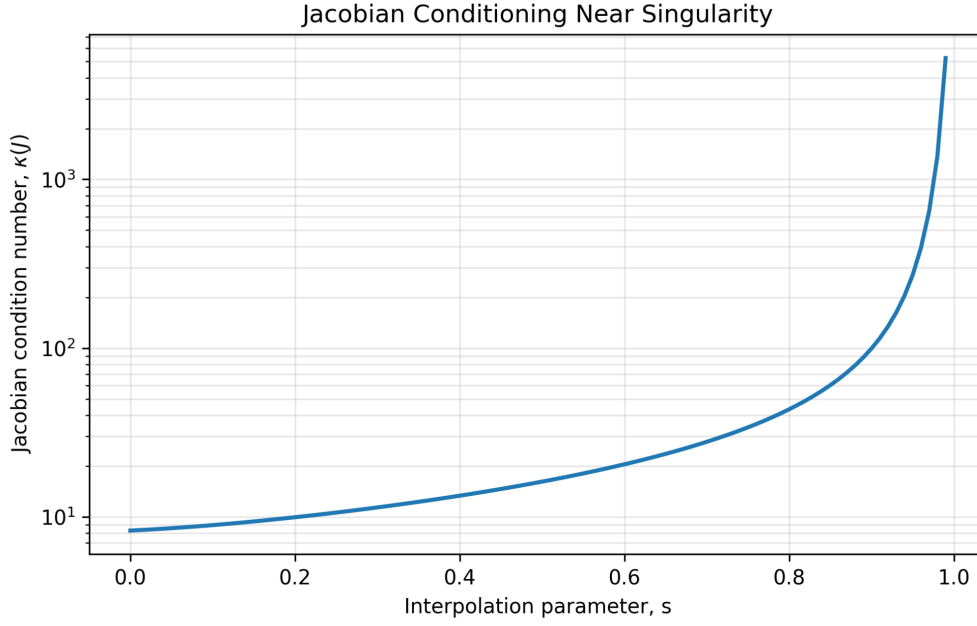


Figure 2: Jacobian condition number on a logarithmic scale. Numerical sensitivity increases sharply before exact rank loss.

which a pseudoinverse is most sensitive:

$$\dot{\mathbf{q}}_{\text{pinv}} = \mathbf{J}^+ \mathbf{v}, \quad (2)$$

$$\dot{\mathbf{q}}_{\text{DLS}} = \mathbf{J}^T \left(\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I} \right)^{-1} \mathbf{v}, \quad \lambda = 0.05. \quad (3)$$

Because the commanded task-space velocity was chosen as a unit vector along the weakest left-singular direction, the plotted DLS joint-rate norm directly follows the scalar gain

$$G_{\text{DLS}}(\sigma_{\min}) = \frac{\sigma_{\min}}{\sigma_{\min}^2 + \lambda^2}. \quad (4)$$

The gain reaches its theoretical maximum when $\sigma_{\min} = \lambda$:

$$G_{\text{DLS},\max} = \frac{1}{2\lambda} = 10 \quad (\lambda = 0.05). \quad (5)$$

The singularity sweep is discrete and therefore need not sample exactly $\sigma_{\min} = 0.05$; an observed numerical peak slightly below 10 is consequently expected.

The result is not that DLS is universally superior. It makes an explicit robustness–accuracy trade: task-space fidelity is sacrificed near singularity to bound inverse amplification.

5. Dynamic Characterization

I characterized the mass matrix, gravity compensation vector, and Coriolis/centrifugal vector at the same three representative configurations. The test velocity was $\dot{\mathbf{q}} = [20, -30, 40, -20, 30, 10]^\circ/\text{s}$.

The mass matrix remained symmetric positive definite at all three configurations. I do not interpret the large condition numbers as dynamic instability; they reflect strongly anisotropic generalized inertia, especially between proximal-arm and wrist directions. I therefore avoid explicitly forming $\mathbf{M}^{-1}$ in implementation and instead solve

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \boldsymbol{\tau} - \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q}) \quad (6)$$

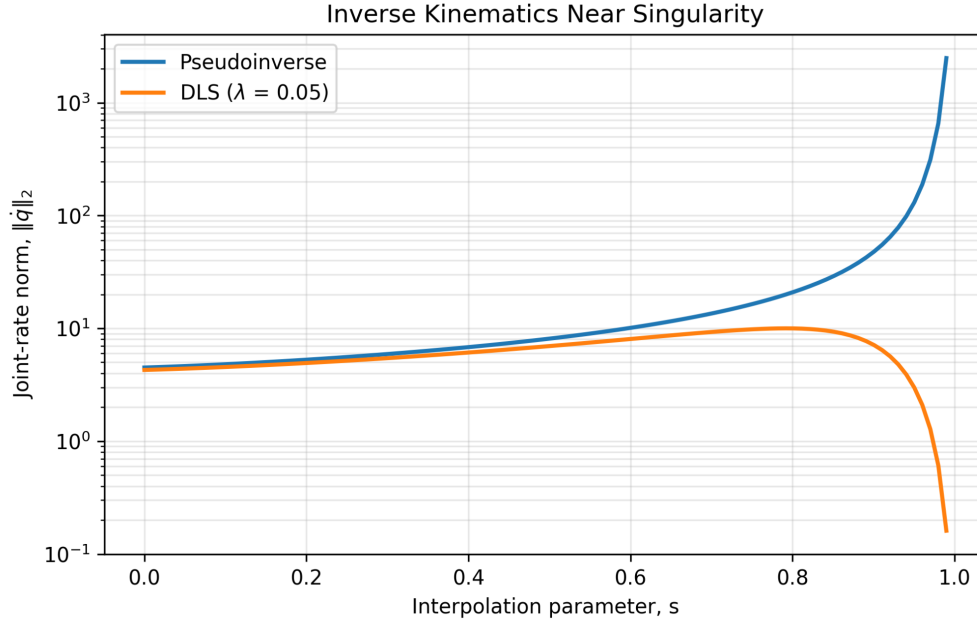


Figure 3: Joint-rate amplification near singularity. The pseudoinverse grows without bound while DLS peaks near the theoretical gain limit and then decreases.

Table 3: Configuration-dependent dynamic quantities.

Configuration	$\lambda_{\min}(\mathbf{M})$	$\lambda_{\max}(\mathbf{M})$	$\kappa(\mathbf{M})$	$\ \mathbf{g}\ $	$\ \mathbf{c}\ $	$\ \mathbf{c}\ /\ \mathbf{g}\ $
Validated	2.55×10^{-4}	3.852	15101	52.862	0.5303	0.0100
Nominal	2.57×10^{-4}	3.016	11713	28.716	0.4021	0.0140
Zero	2.52×10^{-4}	4.685	18587	62.537	0.1581	0.00253

directly as a linear system. For the present 6×6 double-precision model, the observed condition numbers alone do not demonstrate numerical failure, although conditioning remains a quantity to monitor during controller evaluation.

For the specified test velocity, $\|\mathbf{c}\|$ was roughly 0.25–1.4% of $\|\mathbf{g}\|$. This does not justify dropping $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$ from the plant; the result characterizes only this operating condition.

5.1 Coriolis implementation consistency

The Coriolis/centrifugal vector was constructed from numerical derivatives of $\mathbf{M}(\mathbf{q})$ using the Christoffel form. Initial internal checks verified

$$\mathbf{c}(\mathbf{q}, \mathbf{0}) = \mathbf{0} \quad (7)$$

and the expected quadratic velocity dependence

$$\mathbf{c}(\mathbf{q}, 2\dot{\mathbf{q}}) = 4\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}). \quad (8)$$

To strengthen this internal validation, I explicitly constructed the Christoffel-based Coriolis matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ and verified

$$\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}. \quad (9)$$

I also evaluated the standard rigid-body structural identity for

$$\mathbf{N} = \dot{\mathbf{M}}(\mathbf{q}) - 2\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}), \quad (10)$$

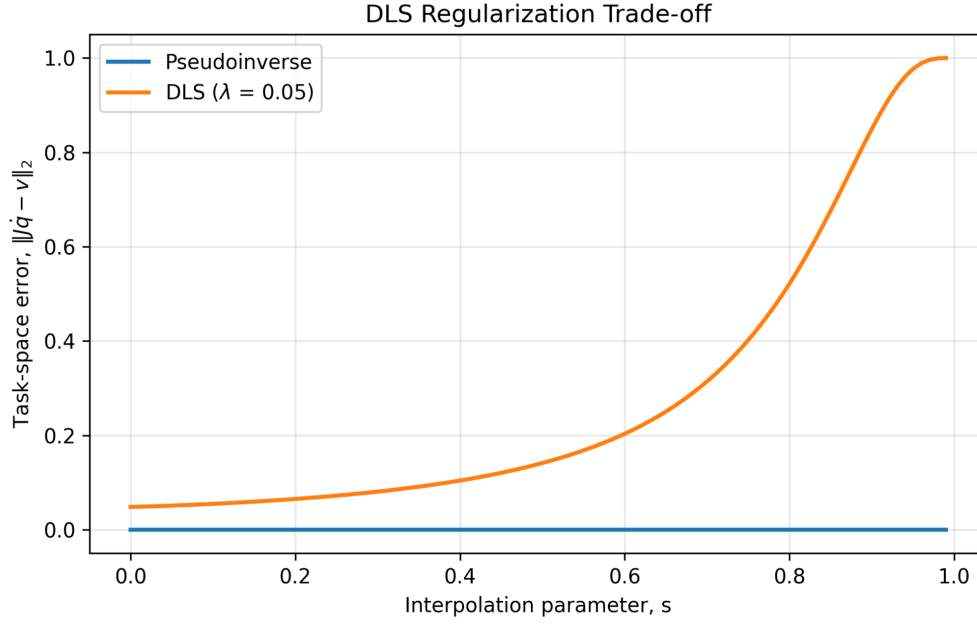


Figure 4: Regularization trade-off. DLS bounds joint-rate amplification by accepting increasing task-space error near singularity.

which requires $\mathbf{N}$ to be skew-symmetric. Numerically, I evaluated

$$\mathbf{R} = \mathbf{N} + \mathbf{N}^\top. \quad (11)$$

Across the validated test pose, nominal working pose, and zero configuration, the maximum discrepancy between $\mathbf{C}\dot{\mathbf{q}}$ and the existing Coriolis/centrifugal vector was

$$\max |\mathbf{C}\dot{\mathbf{q}} - \mathbf{c}| = 8.33 \times 10^{-17}, \quad (12)$$

while the maximum structural residual was

$$\max |\mathbf{R}| = 1.44 \times 10^{-9}. \quad (13)$$

These results provide strong structural-consistency evidence for the numerical Christoffel implementation. They remain internal model checks rather than an independent physics-engine validation.

6. Nonlinear State-Space Plant and Equilibria

The nonlinear torque-driven plant used for controller work is

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}(\mathbf{q})^{-1} (\boldsymbol{\tau} - \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{g}(\mathbf{q})) \end{bmatrix}. \quad (14)$$

In implementation, the lower block is evaluated by solving the corresponding linear system rather than explicitly forming $\mathbf{M}^{-1}$.

For any static configuration $\mathbf{q}_{\text{eq}}$ with $\dot{\mathbf{q}}_{\text{eq}} = \mathbf{0}$, I defined $\boldsymbol{\tau}_{\text{eq}} = \mathbf{g}(\mathbf{q}_{\text{eq}})$. Substitution into the nonlinear state derivative returned exactly zero at the validated, nominal, and zero configurations in the implemented numerical model.

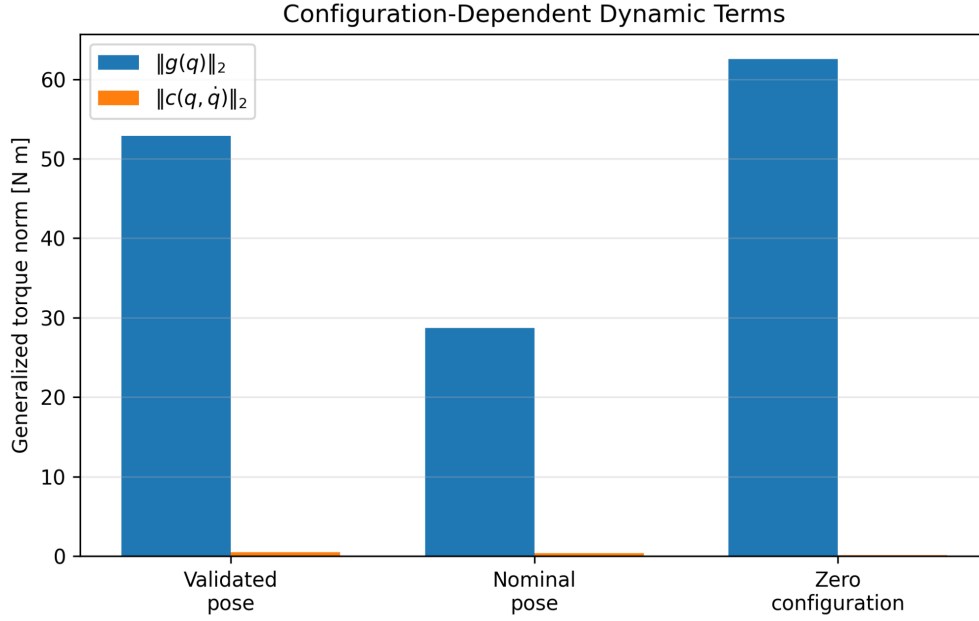


Figure 5: Norms of the gravity and Coriolis/centrifugal generalized torque terms for the specified test velocity.

7. Local Linearization, Controllability, and Open-Loop Structure

I numerically linearized the nonlinear state equation around each static equilibrium using central finite differences. At a static equilibrium in this frictionless model, the expected block structure is

$$A = \begin{bmatrix} 0 & I \\ A_{21} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \mathbf{M}(\mathbf{q}_{\text{eq}})^{-1} \end{bmatrix}. \quad (15)$$

The controllability matrix had rank 12/12 at all three investigated equilibria.

8. Local Validity of the Linearized Dynamics

I perturbed Joint 2 around the nominal equilibrium while holding the equilibrium torque fixed and compared the nonlinear change in $\dot{\mathbf{x}}$ with $A\delta\mathbf{x} + B\delta\boldsymbol{\tau}$.

Table 4: Local linearization error for Joint-2-only perturbations.

Joint 2 perturbation	Relative linearization error
0.01°	0.0188%
0.1°	0.188%
1°	1.91%
5°	9.97%
10°	20.79%

The error increased systematically with perturbation magnitude. I do not impose an arbitrary angle at which the model becomes invalid; acceptable model error depends on the controller objective and robustness margin.

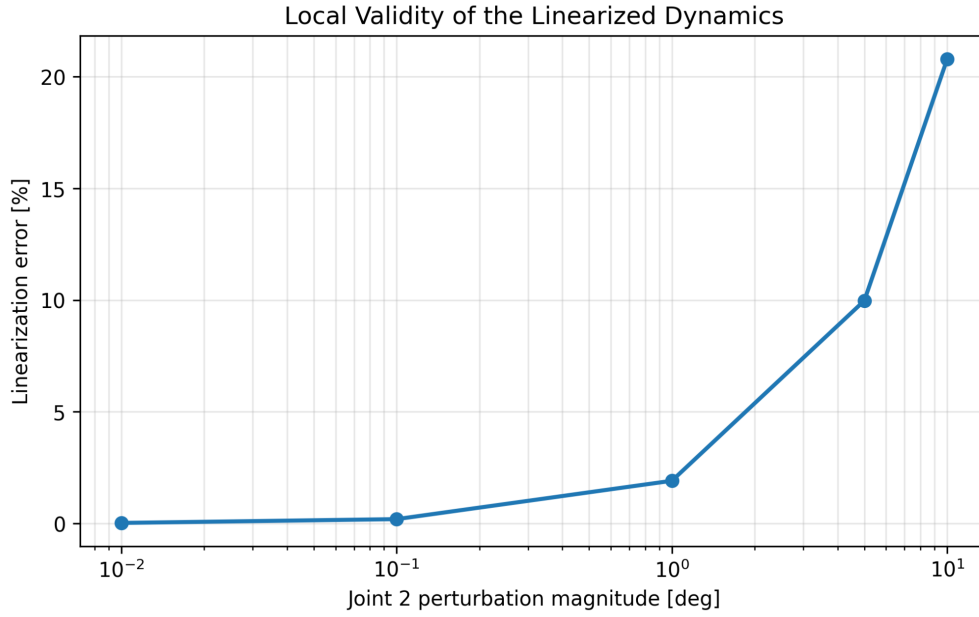


Figure 6: Relative error of the nominal-equilibrium linear model under Joint-2-only perturbations.

9. Common Controller Benchmark Definition

The common reference moves from $\mathbf{q}_0 = [0, -90, 90, -90, -90, 0]^\circ$ to $\mathbf{q}_f = [20, -60, 60, -70, -70, 20]^\circ$ in 4.0 s using the quintic time-scaling law

$$h(s) = 10s^3 - 15s^4 + 6s^5, \quad s = t/T, \quad (16)$$

with zero endpoint velocity and acceleration. The sample time is 0.002 s, giving 2001 samples.

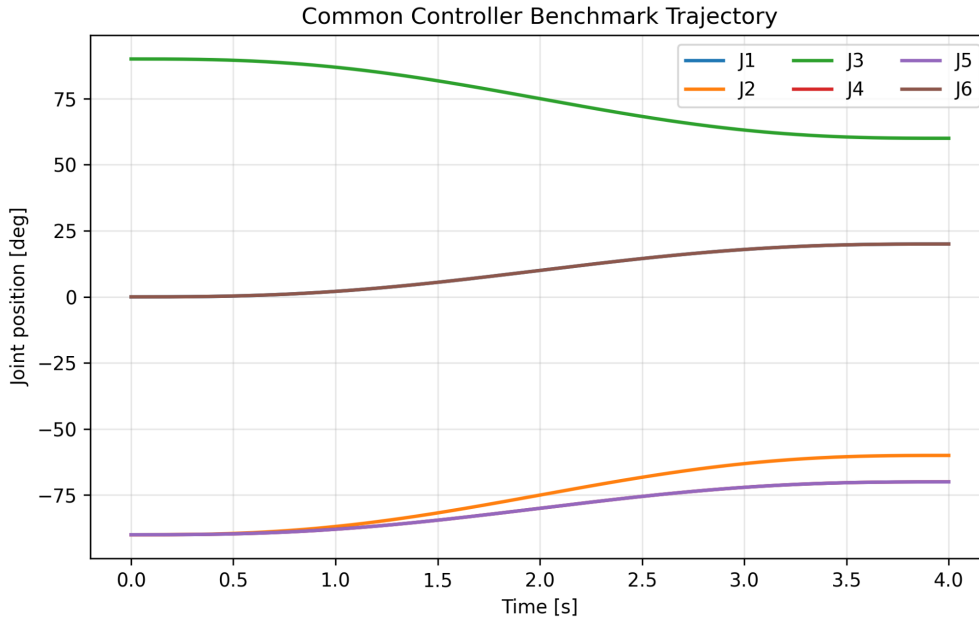


Figure 7: Common 4 s quintic joint-space benchmark used for all Part 1B controllers.

The 2 ms sample time is aligned with the 500 Hz system update frequency listed in the UR5e technical specification. I use that frequency as a benchmark discretization choice; it does not imply that the analytical torque plant reproduces the proprietary internal UR servo.

Table 5: Benchmark reference limits.

Metric	J1	J2	J3	J4	J5	J6
Max $ \dot{q}_d $ [deg/s]	9.375	14.063	14.063	9.375	9.375	9.375
Speed utilization [%]	5.21	7.81	7.81	5.21	5.21	5.21
Max $ \ddot{q}_d $ [deg/s ²]	7.217	10.825	10.825	7.217	7.217	7.217

10. Inverse-Dynamics Feasibility and Simulation Constraints

I computed the nominal feedforward torque

$$\boldsymbol{\tau}_{\text{ff}} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}_d + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}_d) + \mathbf{g}(\mathbf{q}) \quad (17)$$

along the full reference trajectory. The adopted simulation torque constraints are 150 N m for Joints 1–3 and 28 N m for Joints 4–6. These values came from the imported Isaac/URDF model used in the earlier validation stage and are not interpreted as independently verified physical motor-torque ratings.

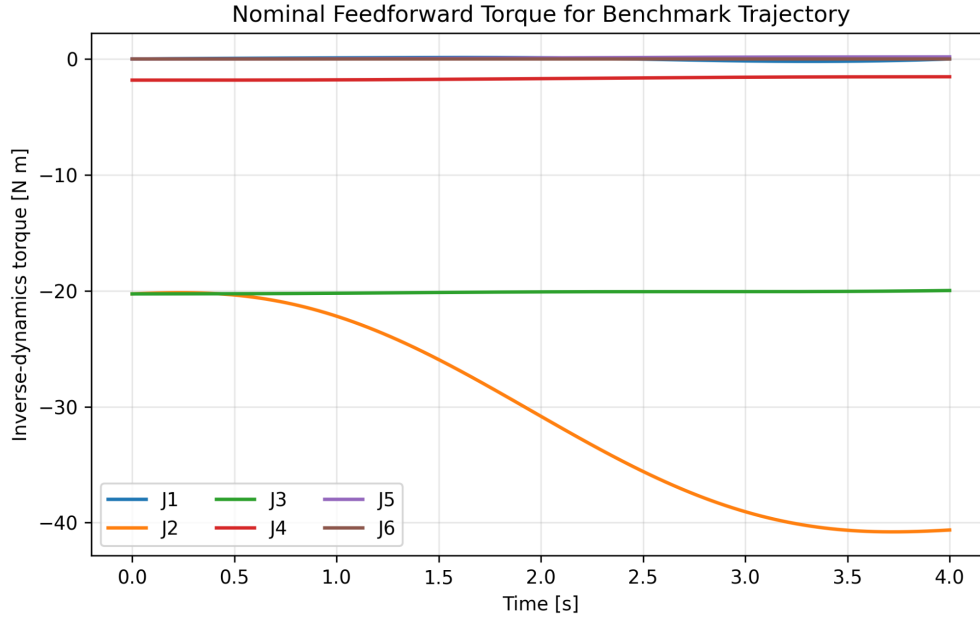


Figure 8: Nominal inverse-dynamics feedforward torque along the common benchmark trajectory.

Table 6: Peak benchmark inverse-dynamics torque.

Joint	Peak $ \tau_{\text{ff}} $ [N m]	Limit [N m]	Utilization	Peak time [s]
J1	0.2063	150	0.138%	3.320
J2	40.7923	150	27.195%	3.720
J3	20.2643	150	13.510%	0.000
J4	1.8253	28	6.519%	0.130
J5	0.1803	28	0.644%	4.000
J6	1.28×10^{-5}	28	$\approx 0\%$	3.352

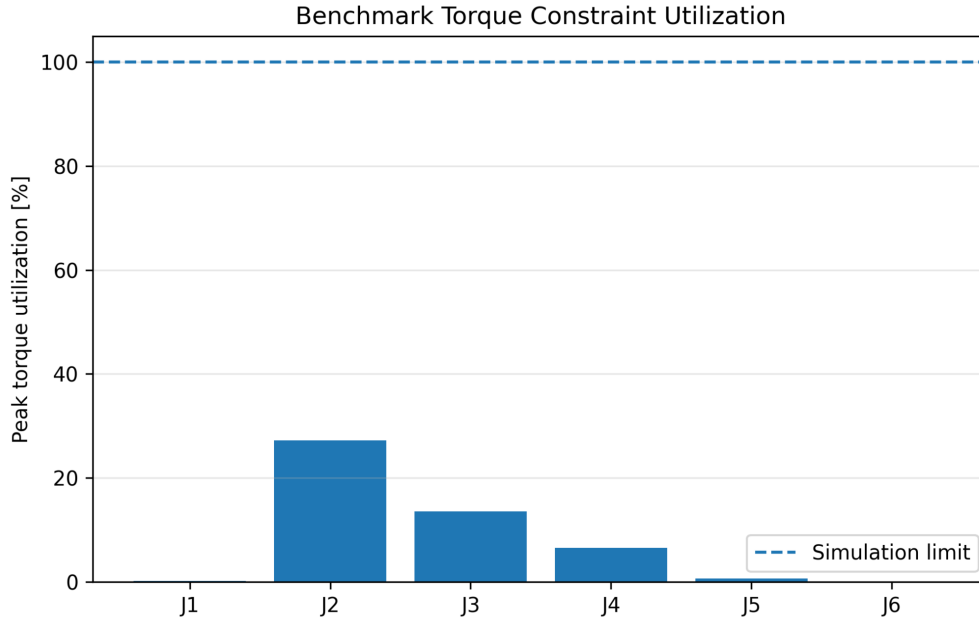


Figure 9: Peak inverse-dynamics torque as a percentage of the adopted simulation constraint.

11. C++ Core Implementation and Cross-Validation

Python remains the numerical reference environment for analysis, plots, parameter studies, and tuning. The runtime core was reproduced in C++ using fixed-size Eigen types. The C++ dynamics code intentionally reproduces the validated physical-link formulation rather than rebuilding dynamics directly from the compact DH chain.

Table 7: Python–C++ numerical cross-validation.

Quantity	Maximum error	Tolerance	Result
TCP position	3.80×10^{-9}	1×10^{-8}	PASS
Jacobian singular values	2.51×10^{-9}	1×10^{-8}	PASS
Jacobian condition number	1.36×10^{-7}	1×10^{-6}	PASS
Mass matrix $\mathbf{M}(\mathbf{q})$	4.93×10^{-10}	1×10^{-8}	PASS
Gravity vector $\mathbf{g}(\mathbf{q})$	2.63×10^{-8}	1×10^{-6}	PASS
Coriolis vector $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$	5.60×10^{-10}	1×10^{-6}	PASS
Trajectory midpoint/endpoints	0 at reported precision	1×10^{-12}	PASS

The overall cross-validation passed. This is a language-level implementation cross-check, not an independent physics-engine validation: the C++ code is a deliberate port of the Python reference formulation. Independent model/engine evidence was obtained earlier through analytical-versus-Isaac comparisons.

12. Engineering Conclusions and Part 1B Design Implications

Part 1A changed the controller-design problem from a generic “tune four controllers” exercise into a set of specific engineering questions. The plant is configuration dependent, singularity sensitivity appears well before exact rank loss, gravity loading is substantial, one nominal linearization is demonstrably local, and the common benchmark is dynamically nontrivial without being saturation dominated.

Table 8: Part 1A observations and Part 1B implications.

Part 1A observation	Part 1B implication
Gravity torque dominates the selected moderate-speed test	A plain joint PID needs sufficient integral action or gravity compensation; computed torque should have a structural advantage in nominal tracking.
$M(\mathbf{q})$ changes strongly with configuration	Fixed gains may produce configuration-dependent closed-loop behavior; model-based controllers can explicitly account for inertia variation.
Linearization error grows with distance from $\mathbf{q}_{\text{eq}}$	LQR performance should be interpreted relative to its design equilibrium; large-motion tracking is also a model-mismatch test.
DLS bounds inverse amplification near singularity	Any future Cartesian command layer should use a robustness strategy rather than a raw pseudoinverse near poor conditioning.
Benchmark torque uses at most 27.2% of the adopted simulation constraint	Nominal comparison should expose controller behavior rather than primarily saturation behavior.
Python and C++ agree within tolerance	Controller algorithms can be prototyped/analyzed in Python and implemented in C++ against a consistent plant core.

With the nonlinear plant model characterized, its operating limits defined, its local linearization validated, and the investigated operating points verified to be controllable, I considered the model sufficiently characterized to begin controller design.

13. Limitations and Reproducibility Boundaries

- The analysis uses an ideal rigid-body, fixed-base, no-payload model. Friction, compliance, actuator electrical dynamics, sensor noise, and the proprietary internal UR servo are not included.
- The full geometric Jacobian mixes translational and rotational units; its scalar condition/manipulability measures are used comparatively under one convention.
- The linearization-validity sweep perturbs Joint 2 only. It does not establish a global validity region over all state directions.
- The Coriolis/centrifugal magnitude comparison is tied to the specified $\dot{\mathbf{q}}$ test vector and should not be generalized to high-speed motion.
- The Coriolis structural-identity check is an internal consistency validation of the implemented rigid-body model, not an independent Isaac Sim validation.
- The 150/150/150/28/28/28 N m torque values are simulation constraints adopted from the imported model for consistency, not independently verified motor ratings.
- Python–C++ agreement validates implementation consistency. The independent external model comparison is the earlier analytical–Isaac Sim cross-validation.

14. Reproducibility Map

Purpose	Script / artifact
Kinematic characterization	<code>analysis/kinematic_characterization.py</code>

Singularity sweep	analysis/singularity_sweep.py	→
	results/data/singularity_sweep.csv	
Pseudoinverse vs DLS	analysis/pseudoinverse_vs_dls.py	→
	results/data/pseudoinverse_vs_dls.csv	
Dynamics characterization	analysis/dynamics_characterization.py	→
	results/data/dynamics_characterization_summary.csv	
Coriolis structural validation	analysis/coriolis_structural_validation.py	→
	results/data/coriolis_structural_validation.csv	
Equilibrium validation	analysis/equilibrium_validation.py	
Linearization / controllability	analysis/linearization_analysis.py	
Linearization validity	analysis/linearization_validation.py	→
	results/data/linearization_validation.csv	
Benchmark reference	analysis/benchmark_trajectory.py	→
	results/data/benchmark_trajectory.csv	
Inverse dynamics	analysis/benchmark_inverse_dynamics.py	→
	results/data/benchmark_inverse_dynamics.csv	
C++ validation	cpp/tests/cross_validation.cpp	→
	results/data/cpp_cross_validation.txt	

References

- [1] Universal Robots, “Denavit-Hartenberg parameters,” UR Developer documentation.
<https://www.universal-robots.com/developer/hardware-and-motion/robot-motion-dh-parameters> (accessed Sep. 2026).
- [2] Universal Robots, “Technical Specifications UR5e,” PolyScope 5 manual documentation.
https://www.universal-robots.com/manuals/EN/HTML/SW5_24/Content/prod-usr-man/complianceUR5e/H_g5_sections/appendix_g5/tech_spec_sheet.htm (accessed Sep. 2026).
- [3] Universal Robots, “Universal_Robots_ROS2_Description,” GitHub repository, `ur_description` package.
https://github.com/UniversalRobots/Universal_Robots_ROS2_Description (accessed Sep. 2026).
- [4] Sooyong Kim, *ur5e-control-analysis*. Project repository containing Python analysis, fixed-size Eigen C++ core, benchmark data, figures, and cross-validation results.

A. Representative Numerical Results

A.1 Validated-pose mass matrix

$$M(q) = \begin{bmatrix} 3.103413 & -0.173015 & 0.093256 & 0.027184 & -0.039033 & 0.000031 \\ -0.173015 & 3.342034 & 1.208672 & 0.046755 & -0.003372 & 0.000182 \\ 0.093256 & 1.208672 & 0.833906 & 0.071411 & -0.011963 & 0.000182 \\ 0.027184 & 0.046755 & 0.071411 & 0.021196 & -0.004035 & 0.000182 \\ -0.039033 & -0.003372 & -0.011963 & -0.004035 & 0.005039 & 0.000001 \\ 0.000031 & 0.000182 & 0.000182 & 0.000182 & 0.000001 & 0.000258 \end{bmatrix}.$$

Eigenvalues:

$$[2.55 \times 10^{-4}, 3.714 \times 10^{-3}, 1.338 \times 10^{-2}, 0.34497, 3.09150, 3.85203].$$

Gravity vector [N m]:

$$[0, -49.68549, -18.03463, -0.70750, 0.06887, 0].$$

Coriolis/centrifugal vector at the test velocity [N m]:

$$[-0.44132, 0.08770, 0.27804, 0.03795, -0.00309, 0.000033].$$

A.2 Coriolis structural-validation record

For the three investigated configurations,

$$\max |\mathbf{C}\dot{\mathbf{q}} - \mathbf{c}| = 8.33 \times 10^{-17}, \quad \max |(\dot{\mathbf{M}} - 2\mathbf{C}) + (\dot{\mathbf{M}} - 2\mathbf{C})^\top| = 1.44 \times 10^{-9}.$$

The detailed numerical record is stored in `results/data/coriolis_structural_validation.csv`.

A.3 C++ cross-validation record

Overall result: **PASS**. The complete machine-readable/archival console output is stored in `results/data/cpp_cross_validation.txt` in the project repository.